

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO4: Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 15

Total Marks:30

Annexure A

CLASS TEST

Code of Conduct:

I ____Rashmi Naurene (192521357)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***rashmi***

RUBRICS FOR CALCULATION:

CLASS TEST

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding ; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1

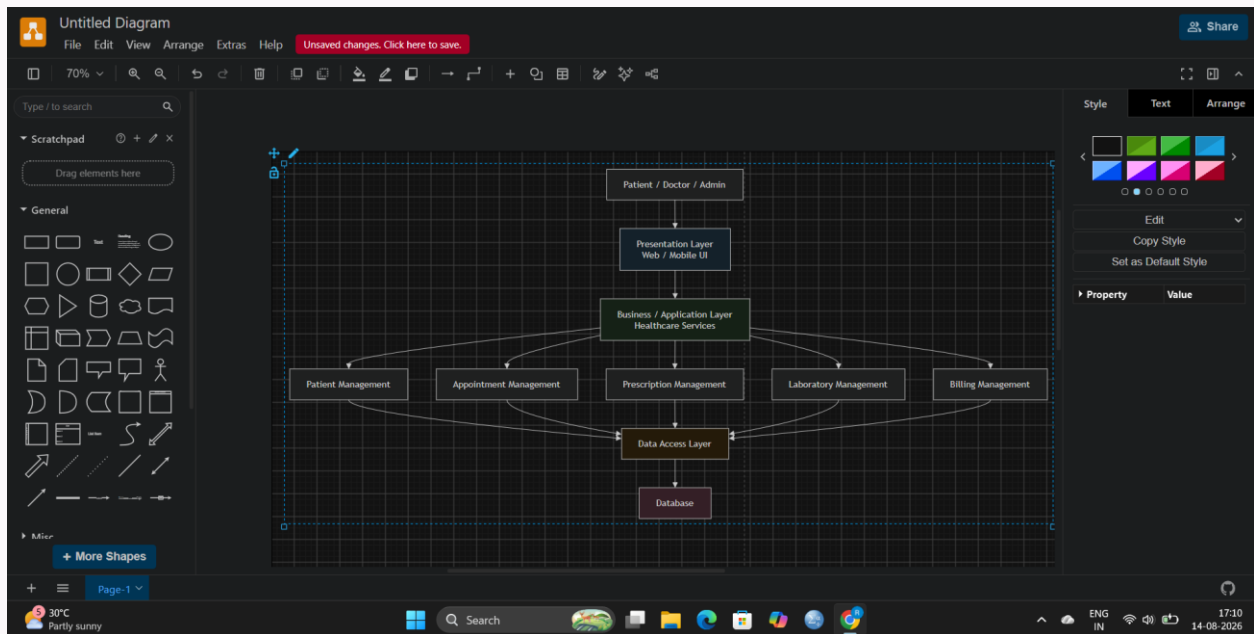
Marks: 2 | Bloom's Level: L1, L2, L3

A Smart Healthcare Application is being developed to manage patients, appointments, prescriptions, laboratory reports, and billing.

Question: Define architectural patterns in object-oriented systems. Identify any two common architectural patterns and explain how Layered Architecture can improve modularity, scalability, and maintainability in the healthcare application.

Answer

An architectural pattern is a reusable, high-level solution describing how a system's major components are organised and how they interact. Two common patterns are Layered (N-Tier) Architecture and Microservices Architecture. In the healthcare app, separating Presentation, Business, and Data layers lets each be changed, tested, and scaled independently — e.g., fixing a billing bug or adding servers at peak hours never touches the database or patient-facing UI.



Question 2

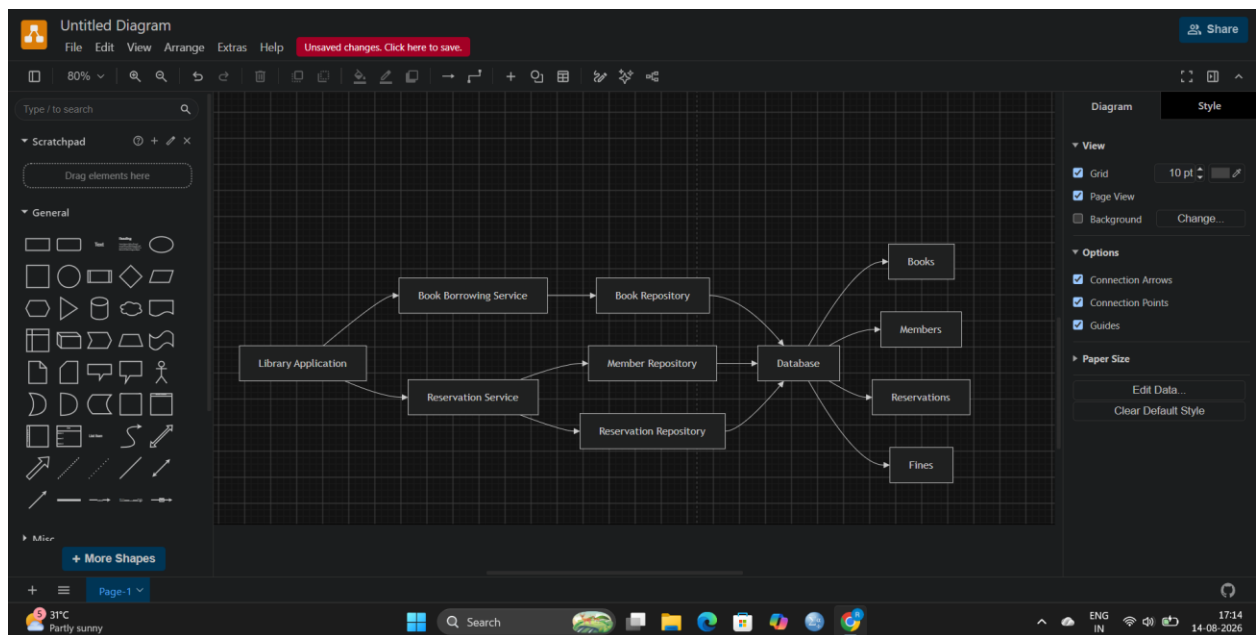
Marks: 2 | Bloom's Level: L2, L3, L4

An Online Library System stores information about books, members, borrowing transactions, reservations, and fines. Developers want to isolate database operations from the application logic.

Question: What is the Repository Pattern? Mention two advantages and explain how it can separate data-access operations from book-borrowing and reservation logic.

Answer

The Repository Pattern is an abstraction layer that exposes collection-like methods (Add, FindById, Update) for accessing domain objects, hiding the actual database queries behind them. Two advantages: it decouples business logic from the storage technology, and it improves testability via mock repositories. A BookRepository and ReservationRepository handle all SQL, so the BorrowingService only calls their methods and stays focused purely on borrowing rules.



Question 3

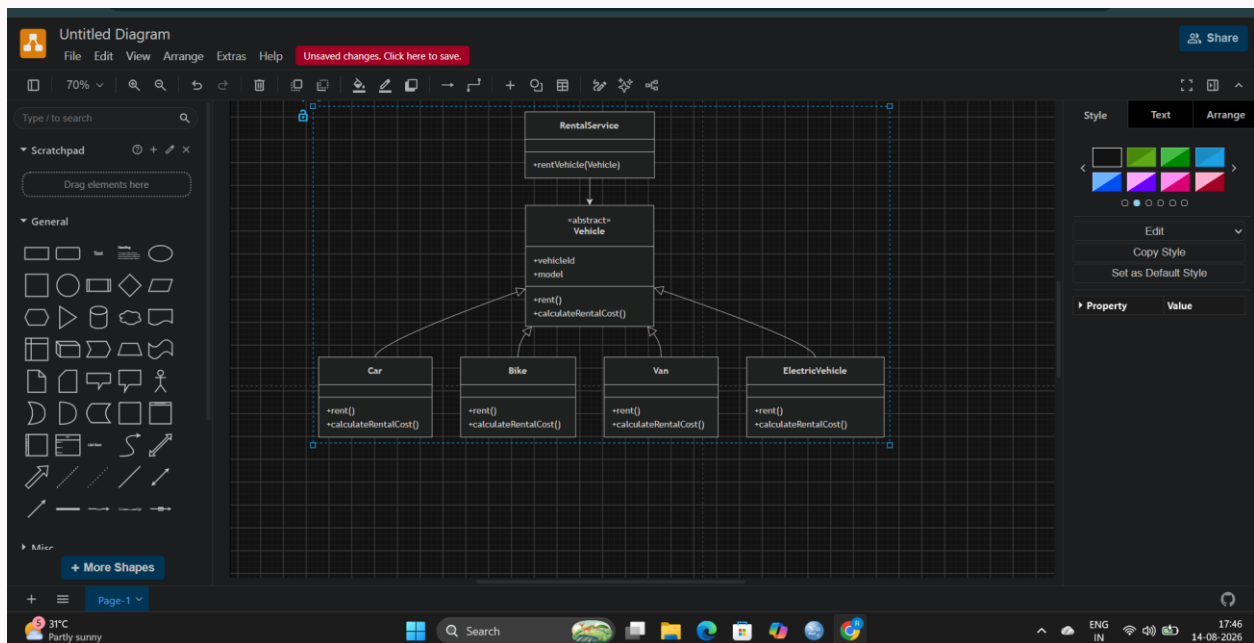
Marks: 2 | Bloom's Level: L2, L3, L4

A Vehicle Rental Application supports cars, bikes, vans, and electric vehicles. The system should allow new vehicle types to be introduced without breaking existing functionality.

Question: What are object-oriented design guidelines? Identify any two categories and explain how the Open-Closed Principle (OCP) supports extensibility and reusable components in the vehicle rental system.

Answer

Object-oriented design guidelines are established principles that keep class structures understandable and resilient to change; two categories are SOLID Principles and Coupling/Cohesion Guidelines. OCP states code should be open for extension but closed for modification. Here, an abstract Vehicle type is used by the rental engine, so a new type like Scooter is added as a new subclass without editing or re-testing existing rental code.



Question 4

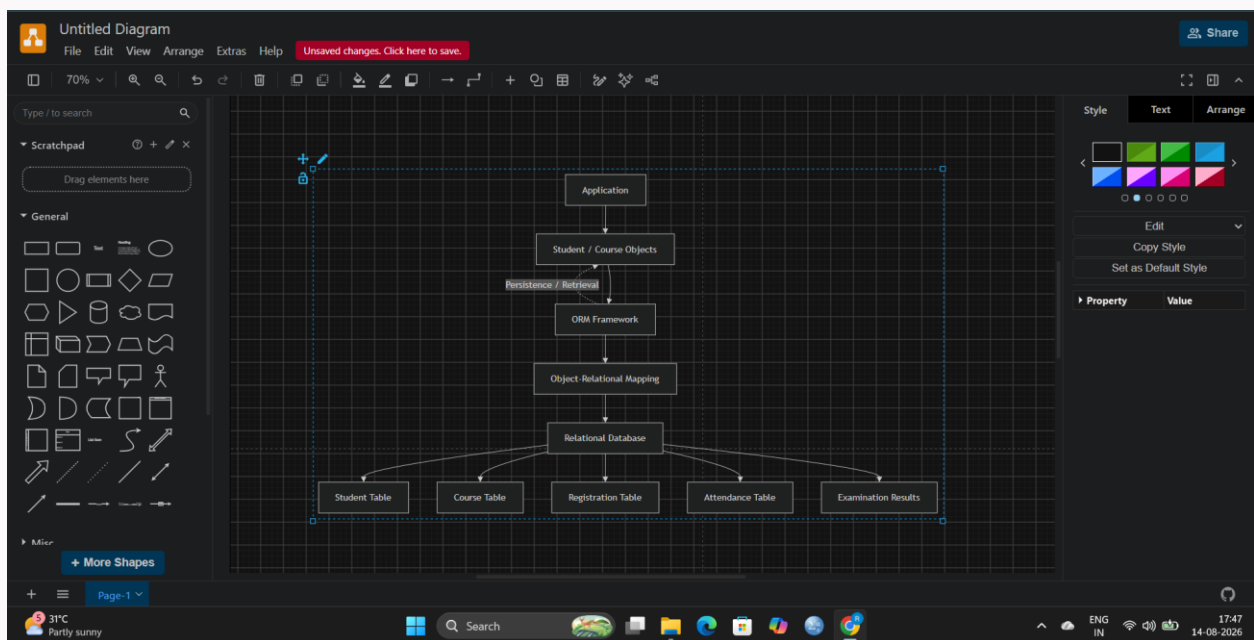
Marks: 2 | Bloom's Level: L1, L2, L3

A University Management System stores student profiles, course registrations, attendance, examination results, and faculty information. These objects must remain available after the application is closed.

Question: Define object persistence in an object-oriented application. Mention two persistence mechanisms and explain how an Object-Relational Mapping (ORM) framework can manage persistent student and course objects.

Answer

Object persistence is an object's ability to have its state survive beyond the application process, by saving it to non-volatile storage and reconstructing it later. Two mechanisms: file-based serialization and relational storage via ORM. An ORM (e.g., Hibernate) maps Student and Course objects to STUDENT/COURSE tables and their enrolments to a join table, generating the SQL automatically so developers work with objects, not queries.



Question 5

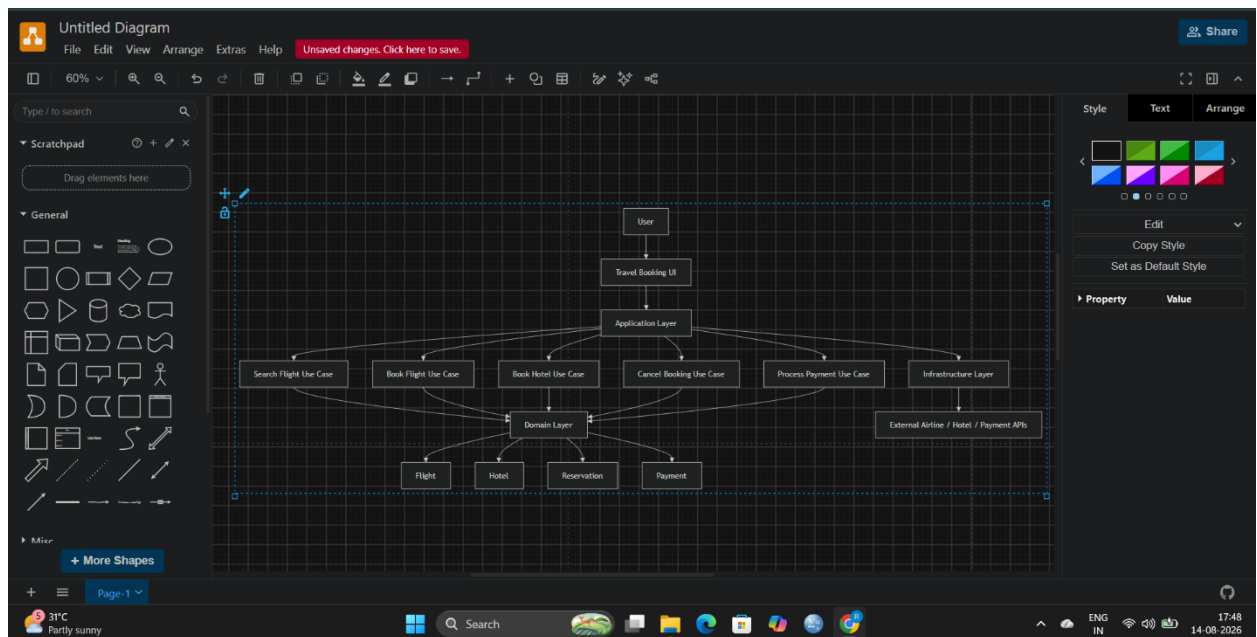
Marks: 2 | Bloom's Level: L2, L3, L4

An Online Travel Booking System allows users to search flights and hotels, make reservations, process payments, and cancel bookings. Complex business rules must be coordinated without being placed directly inside domain entities.

Question: What is an Application Layer? State two major responsibilities and explain how it coordinates travel-booking use cases without directly implementing core domain business rules.

Answer

The Application Layer sits between Presentation and Domain layers, orchestrating use-case steps without containing the actual business rules. Its two responsibilities are coordinating multi-step workflows and managing transaction boundaries. A BookingApplicationService calls FlightService, HotelService, and PaymentService in sequence, but the rule “no confirmation without payment” itself lives inside the Booking domain entity, not the application service.



Question 6

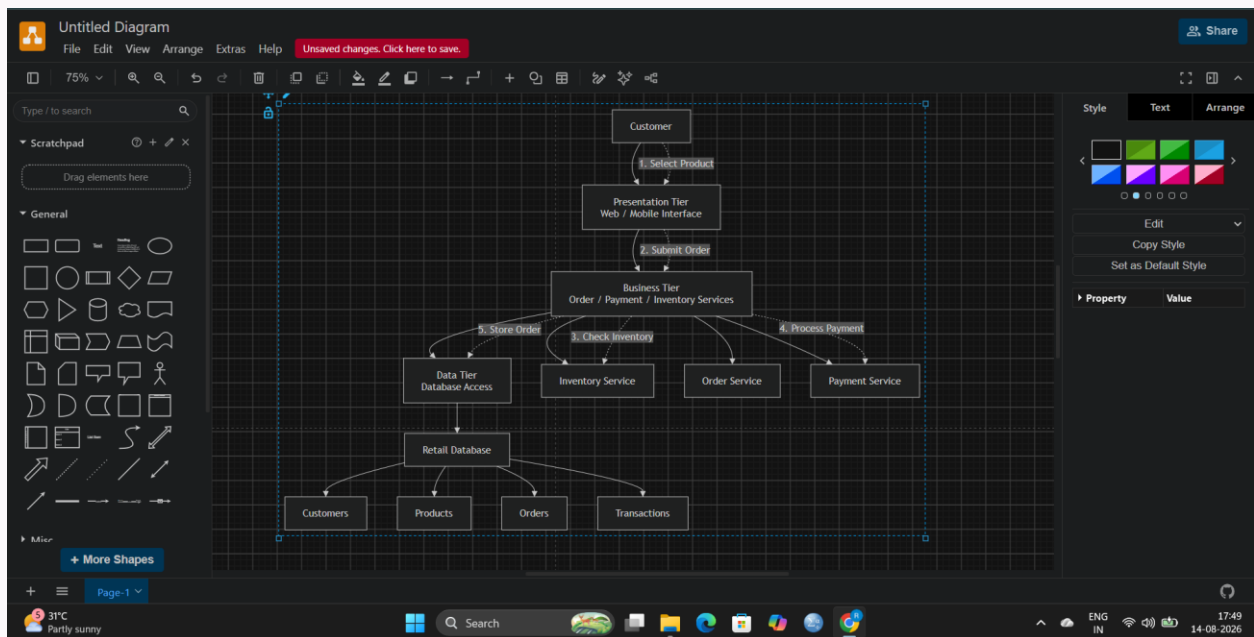
Marks: 3 | Bloom's Level: L2, L4, L5

An Online Retail System consists of a web/mobile interface, business services for orders and payments, and a database containing customer, product, and transaction information.

Question: Explain the structure of a three-tier architecture consisting of Presentation, Business, and Data layers. Analyze the flow of an online purchase from product selection through payment and order storage.

Answer

Presentation handles UI only, Business implements order/pricing/payment rules, and Data stores customer, product, and transaction records — each layer talking only to the one adjacent to it. For a purchase: Presentation sends the cart request to Business, which checks stock and price, calls PaymentService to authorise payment, then instructs Data to insert the order and update inventory before confirmation flows back to the user.



Question 7

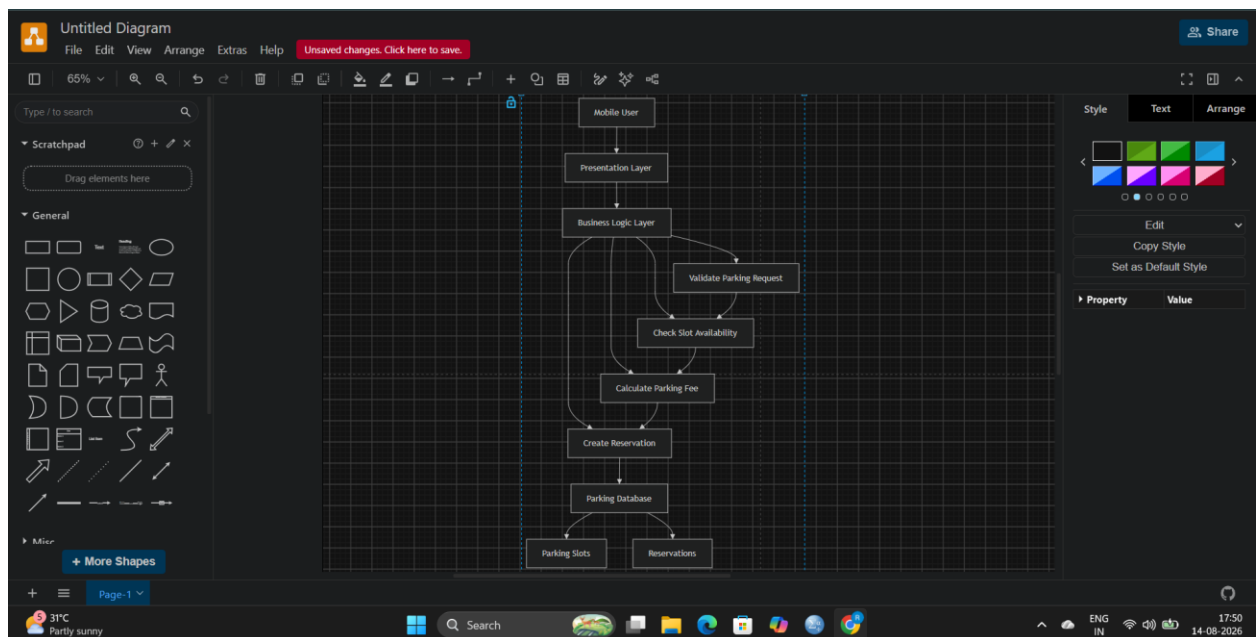
Marks: 2 | Bloom's Level: L2, L3, L4

A Smart Parking Application receives parking requests from mobile users and checks slot availability, calculates parking fees, and confirms reservations.

Question: What is the role of the Business Logic Layer? Identify two important functions and explain how it validates and processes parking requests received from the presentation layer.

Answer

The Business Logic Layer implements the system's core rules and calculations, sitting between Presentation and Data. Two key functions: validating requests against business rules, and performing calculations like fee computation. When a request arrives from the Presentation Layer, the BLL checks slot availability against the Data Layer, applies pricing rules, and returns a confirmation or rejection — the Presentation Layer never queries the database directly.



Question 8

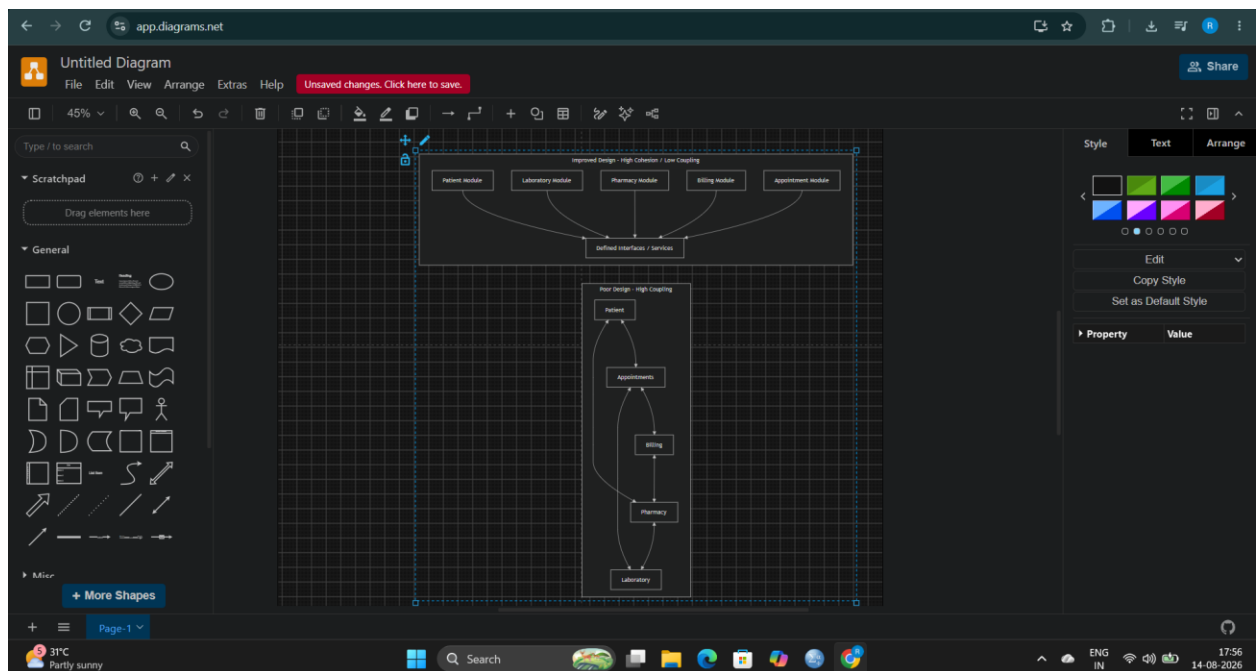
Marks: 3 | Bloom's Level: L3, L4, L5

A Smart Hospital System contains separate modules for patient management, appointments, billing, pharmacy, and laboratory services. Some modules have become highly dependent on one another.

Question: Explain the significance of cohesion and coupling in object-oriented design. Analyze how high cohesion and low coupling can improve maintainability, reusability, and flexibility in the hospital system.

Answer

Cohesion is how focused a module's responsibilities are; coupling is how dependent modules are on each other's internals. The hospital's tightly interdependent modules show low cohesion/high coupling. If Billing stays focused only on billing (high cohesion) and talks to Pharmacy or Lab only through defined interfaces (low coupling), changes stay contained — improving maintainability, letting modules be reused elsewhere, and allowing any one module to be replaced without breaking the rest.



Question 9

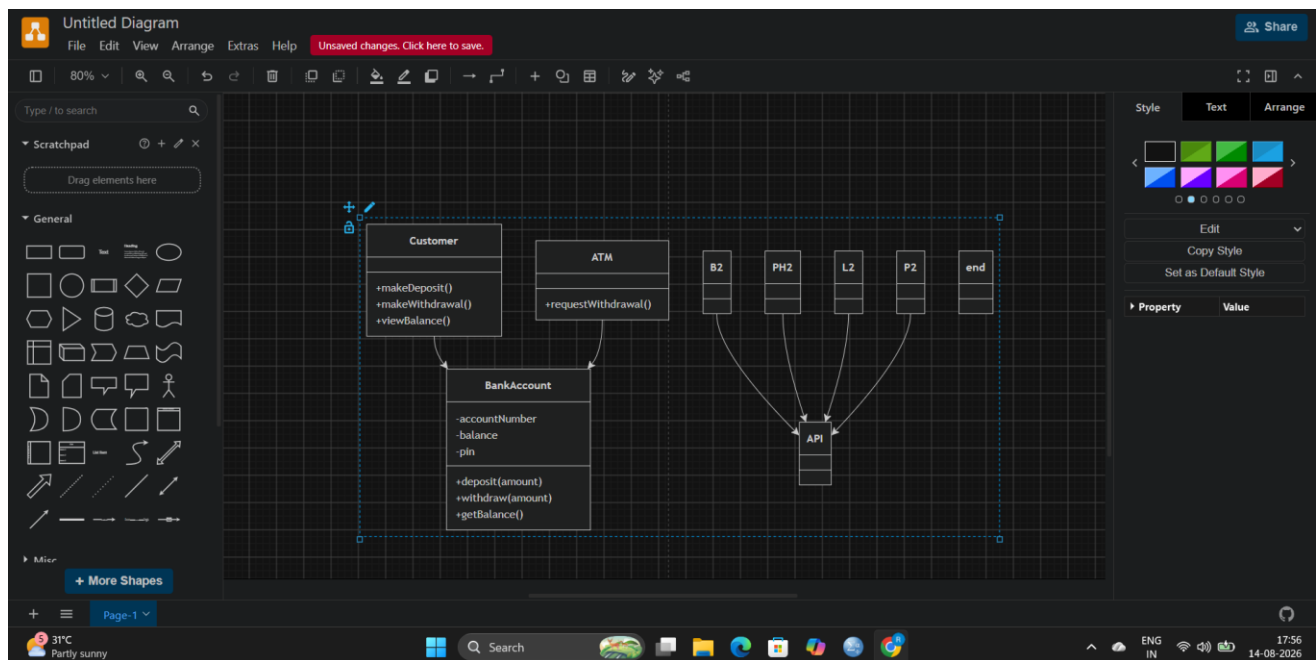
Marks: 2 | Bloom's Level: L2, L4

A Digital Banking Application manages customer accounts and transactions. Account balances should not be modified directly by external classes.

Question: Define encapsulation in object-oriented systems. Explain how encapsulation protects account state and distinguish it from data hiding using a suitable banking example.

Answer

Encapsulation bundles an object's data with the methods that operate on it, controlling external access. In the Account class, balance is private and can only change via withdraw()/deposit(), which validate the amount first — preventing invalid states like a negative balance. Data hiding is the narrower outcome (concealing balance specifically via a private modifier), while encapsulation is the broader technique of bundling data and behaviour together.



Question 10

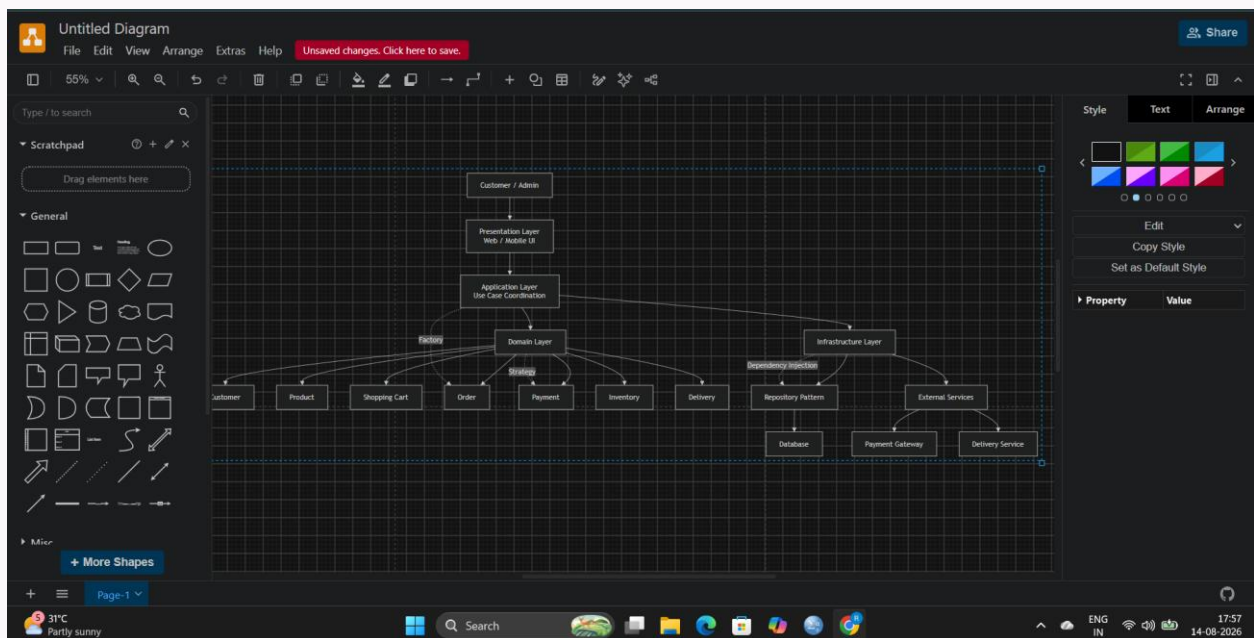
Marks: 4 | Bloom's Level: L3, L4, L5

A Smart E-Commerce Application manages customers, products, shopping carts, orders, payments, inventory, and delivery services. The application must support a large number of users and protect sensitive transaction information.

Question: Design a suitable layered object-oriented architecture for the e-commerce application. Identify the major layers, recommend appropriate design patterns, and justify how the architecture improves security, scalability, maintainability, and flexibility.

Answer

Five layers fit best: Presentation (UI/API), Application (orchestrates PlaceOrderService), Domain (Product, Cart, Order, Payment + business rules), Data Access (Repository per aggregate), and Infrastructure (payment gateway, notifications). Recommended patterns: Repository, Strategy (payment methods), Factory (order types), Observer (notify inventory on order), and Singleton (shared config). Security improves since Presentation never touches the database directly; scalability comes from stateless Presentation/Application layers; maintainability and flexibility come from isolated layers and OCP-friendly patterns that let new features be added as new classes.



Question 11

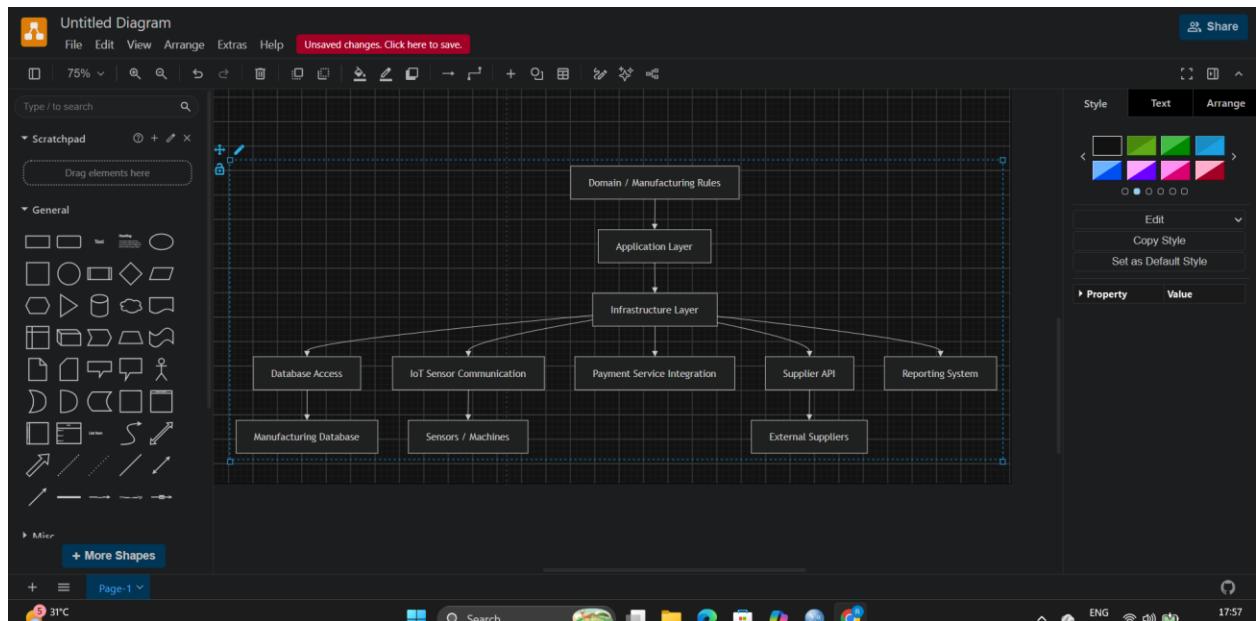
Marks: 2 | Bloom's Level: L1, L2, L3

A Smart Manufacturing System communicates with databases, IoT sensors, payment services, external suppliers, and reporting systems. These technical operations should not be mixed with core manufacturing rules.

Question: What is the role of the Infrastructure Layer in object-oriented architecture? List two responsibilities and explain how it supports database access, IoT communication, and integration with external systems.

Answer

The Infrastructure Layer handles technical and cross-cutting concerns — databases, external services, hardware — keeping them out of the business logic. Two responsibilities: implementing interfaces defined by upper layers, and managing technical concerns like connection pooling and logging. It hosts the DB repository implementations, an MQTT/OPC-UA adapter for sensors, and supplier API clients, so the manufacturing domain layer only calls abstract interfaces, unaware of the underlying protocol.



Question 12

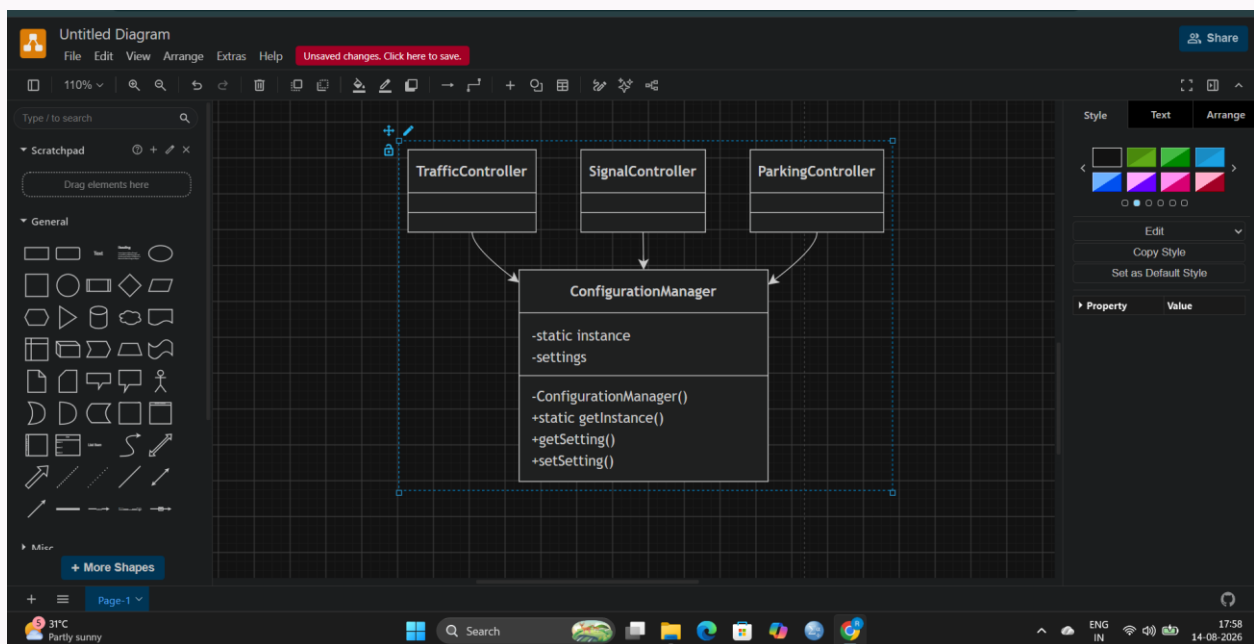
Marks: 2 | Bloom's Level: L2, L3, L4

A Smart Traffic Management System requires a centralized configuration manager to maintain common system settings used by multiple application components.

Question: Define the Singleton Design Pattern. Mention its major characteristics and explain one suitable use of Singleton for shared resources. Also discuss one limitation or risk associated with excessive use of the pattern.

Answer

Singleton restricts a class to exactly one instance, accessed globally via a static method, with a private constructor preventing external creation. A ConfigurationManager Singleton is ideal for shared signal-timing settings, ensuring every module reads identical values. Risk: overuse creates hidden global state, raising coupling and making unit testing difficult since the instance can't easily be mocked or reset between tests.



Question 13

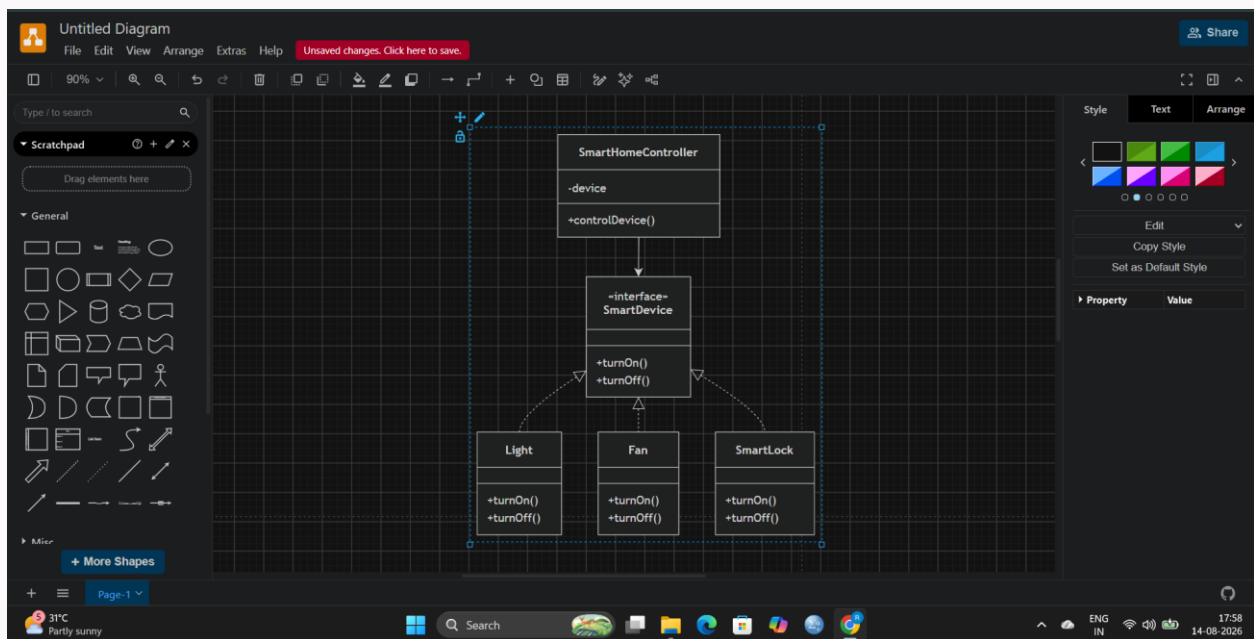
Marks: 2 | Bloom's Level: L2, L3, L4

A Smart Home Automation System controls lights, fans, security cameras, thermostats, and smart locks. The application should work with different device implementations without directly depending on concrete device classes.

Question: What is Dependency Injection in object-oriented design? Mention two benefits and analyze how dependency injection reduces tight coupling and improves unit testing in the smart home system.

Answer

Dependency Injection supplies a class's dependencies from outside (usually via constructor) instead of the class creating them itself. Two benefits: reduced coupling via abstractions, and easier unit testing via mocks. SmartLightController depends only on an ILightDevice interface, so new bulb brands are added without changing the controller, and tests can inject a MockLightDevice instead of real hardware.



Question 14

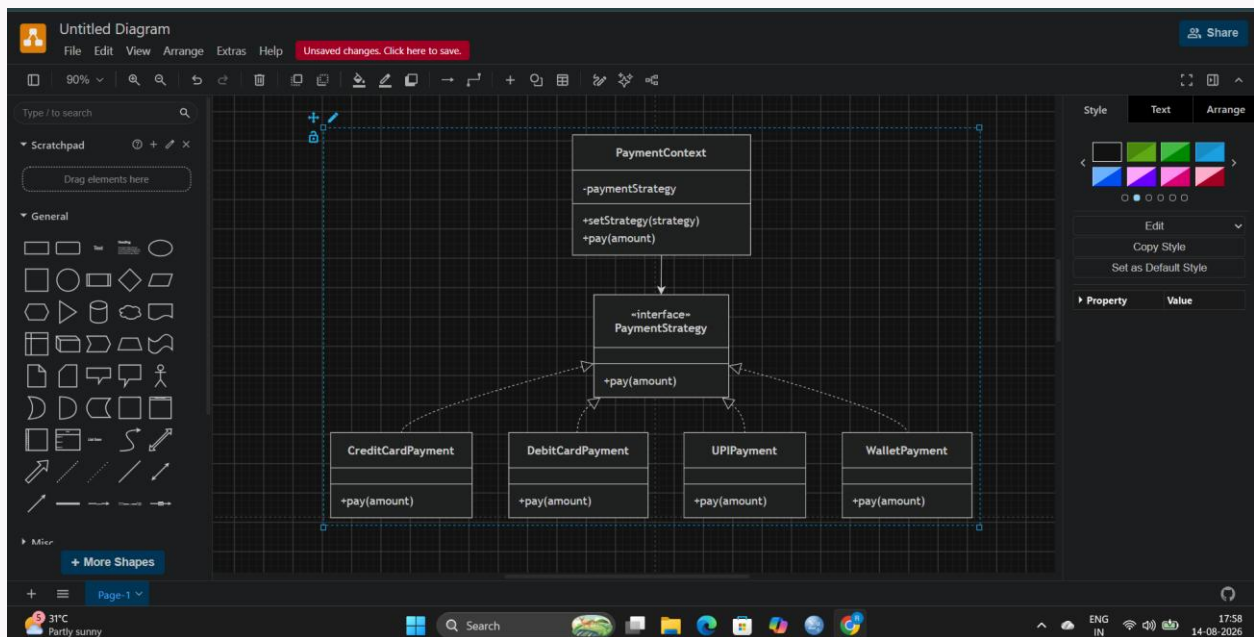
Marks: 3 | Bloom's Level: L3, L4, L5

An Online Payment System supports multiple payment strategies such as credit card, debit card, UPI, and digital wallet payments. The payment method may change according to customer selection.

Question: Explain the working of the Strategy Design Pattern. Compare the roles of Context and Strategy, and illustrate how the pattern can be applied to implement different payment methods.

Answer

Strategy defines a family of interchangeable algorithms, each in its own class behind a common interface, swappable at runtime. Context (PaymentProcessor) holds and calls a Strategy without knowing which one; Strategy (PaymentStrategy) is the interface each payment class implements. Selecting UPI configures PaymentProcessor with UPIPayment instead of CreditCardPayment — the Context's code never changes, and new methods like BNPL are added as new classes only.



Question 15

Marks: 4 | Bloom's Level: L3, L4, L5

A Smart Pharmacy Management System manages customers, medicines, prescriptions, inventory, suppliers, billing, and online orders. The organization expects new services and medicine-management features to be added in the future.

Question: Design a suitable layered architecture for the pharmacy management system. Identify the major layers, recommend relevant design patterns, and justify how the proposed architecture supports flexibility, scalability, maintainability, and future extension.

Answer

The same five layers apply: Presentation, Application (PlaceOrderService, PrescriptionVerificationService), Domain (Medicine, Prescription, Inventory, Supplier), Data Access (Repository per aggregate), and Infrastructure (supplier APIs, notifications). Recommended patterns: Repository, Observer (notify suppliers when stock is low), Strategy (payment/delivery options), Factory (order types), and Singleton (inventory-threshold config). Layering gives flexibility and scalability by isolating change to single layers, and the Observer pattern in particular lets future modules like analytics subscribe to existing stock/order events without modifying Inventory or Order classes at all.

